Ejercicio Demostrativo

Auxiliar: xxxx

Tiempo estimado: 20 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Estructura de Datos - xx]

1. Marco Formativo

1.1 Valores

Orden	En un grafo no dirigido cada arista debe registrarse en ambos extremos y cada vértice visitado debe marcarse. El estudiante que anota cada paso en el pizarrón evita errores y domina la estructura.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Maneja grandes volúmenes de datos aplicando grafos para modelar relaciones entre elementos.
Competencia Específica	Representa un grafo no dirigido con lista de adyacencia; calcula el grado de cada vértice; ejecuta recorridos BFS y DFS; y elimina aristas verificando la conectividad del grafo.

2. Palabras Clave

Grafo no dirigido: Conjunto de vértices unidos por aristas sin dirección: si existe la arista A–B, se puede ir de A a B y de B a A.

Grado de un vértice: Cantidad de aristas que inciden en él. La suma de todos los grados es el doble del número de aristas.

Lista de adyacencia: Representación donde cada vértice guarda la lista de sus vecinos. En un grafo no dirigido cada arista aparece dos veces.

BFS y DFS: Recorridos del grafo. BFS visita por niveles usando una cola; DFS se adentra lo más posible usando recursión o una pila. Ambos usan un arreglo de visitados.

Componente conexa: Conjunto de vértices alcanzables entre sí. Un grafo es conexo si tiene una sola componente.

3. Resumen

Esta sesión introduce los grafos no dirigidos, estructuras que modelan relaciones simétricas entre elementos. El estudiante ya conoce arreglos, listas y árboles; ahora representa un grafo con lista de adyacencia y lo recorre con BFS y DFS.

La clase se orienta a que el estudiante calcule grados, construya la lista de adyacencia, ejecute ambos recorridos y comprenda cómo la eliminación de una arista puede separar el grafo.

4.Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

El ejercicio consiste en construir un grafo no dirigido con vértices A a F y aristas `[A-B, A-C, B-D, C-D, D-E, E-F]`. Se calcula el grado de cada vértice, se arma la lista de adyacencia y se recorre con BFS y DFS desde A. Al final se elimina la arista D–E y se verifica cómo cambia la conectividad.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

> Demostrar cómo se representa un grafo no dirigido con lista de adyacencia y cómo se recorre con BFS y DFS, con el fin de que el estudiante comprenda que cada arista se guarda en ambos extremos y que eliminarla puede dividir el grafo en varias componentes conexas.

4.3 RECURSOS NECESARIOS

- **Software:** Java JDK 11+, IDE (IntelliJ IDEA / Eclipse / VS Code)
- **Herramientas técnicas:** Compilador javac, terminal/consola
- **Materiales de apoyo:** Diapositivas de la semana, pizarrón, visualizador online https://visualgo.net/en/graphds
- **Archivos:** Archivo base `Grafo.java` con la lista de adyacencia ya declarada
- **Configuración mínima:** Java 11+, 4 GB RAM, proyector

4.4 DESARROLLO PASO A PASO

Paso 1 — Grafo y grado de cada vértice (5 min)

Dibujar en el pizarrón el grafo `[A-B, A-C, B-D, C-D, D-E, E-F]`. Pedir a los estudiantes que den el grado de cada vértice sin que lo calcule el auxiliar:

- **Grados esperados:** `A=2  B=2  C=2  D=3  E=2  F=1`
- **Suma de grados:** `12 = 2 × 6 aristas`
- **Vértice de mayor grado:** `D (3)`

Qué observar: Cada arista cuenta dos veces, una por cada extremo; por eso la suma de los grados siempre es el doble del número de aristas.

Paso 2 — Lista de adyacencia (5 min)

Construir la lista insertando las seis aristas; en un grafo no dirigido se agrega en ambos sentidos:

java

```
void agregarArista(int u, int v) {
    lista[u].add(v);
    lista[v].add(u); // no dirigido: ida y vuelta
}
```

```
Lista de adyacencia resultante:
A: B, C
B: A, D
C: A, D
D: B, C, E
E: D, F
F: E
```

Qué observar: Cada arista aparece dos veces (12 entradas en total); el espacio es O(V + E).

Paso 3 — Recorridos BFS y DFS desde A (5 min)

Recorrer el grafo visitando los vecinos en el orden en que aparecen en la lista. BFS usa una cola:

java

```
void bfs(int inicio) {
    boolean[] visitado = new boolean[n];
    Queue<Integer> cola = new LinkedList<>();
    visitado[inicio] = true; cola.add(inicio);
    while (!cola.isEmpty()) {
        int u = cola.poll(); // visita u
        for (int v : lista[u])
            if (!visitado[v]) { visitado[v] = true; cola.add(v); }
    }
}
```

Procedimiento:

1. Cola inicial `[A]`. Sacar A → encolar B y C.
2. Sacar B → encolar D (A ya visitado). Sacar C → D ya está visitado, no se encola.
3. Sacar D → encolar E. Sacar E → encolar F. Sacar F → la cola queda vacía.

Resultado: Orden de visita de cada recorrido:

```
BFS: A B C D E F
```

```
DFS: A B D C E F
```

Qué observar: El grafo tiene el ciclo A–B–D–C–A; sin el arreglo de visitados el recorrido nunca terminaría. DFS baja hasta F antes de regresar, BFS visita por niveles.

Paso 4 — Eliminar una arista y verificar conectividad (5 min)

Eliminar la arista **D–E** y volver a ejecutar BFS desde A:

Procedimiento:

1. Quitar **E** de la lista de D.
2. Quitar **D** de la lista de E. Si se olvida este paso, D ya no ve a E pero E sigue viendo a D y el grafo queda inconsistente.
3. Ejecutar BFS desde A: se visitan A, B, C y D; E y F quedan sin visitar.

Resultado: El grafo queda con **2 componentes conexas**: `{A, B, C, D}` y `{E, F}`.

Qué observar: D–E era un puente: la única arista que unía ambas partes. Un recorrido desde un vértice solo alcanza su propia componente.

Observaciones Importantes

- En un grafo no dirigido, B está en la lista de A si y solo si A está en la lista de B; toda alta o baja de arista debe hacerse en ambas listas.
- BFS encuentra caminos con el menor número de aristas; DFS explora en profundidad y sirve para detectar ciclos y componentes conexas.
- Un grafo conexo con V vértices y exactamente V − 1 aristas es un árbol; si tiene más aristas, contiene al menos un ciclo, como el de este ejercicio (6 vértices y 6 aristas).

4.5 Tabla Comparativa — Lista vs Matriz de Adyacencia

TABLA COMPARATIVA

Operación	Lista	Matriz	Mejor opción	Observación
Espacio	O(V + E)	O(V²)	Lista	Ventaja en grafos con pocas aristas
¿Existe arista u–v?	O(grado(u))	O(1)	Matriz	Acceso directo por índices
Listar vecinos de u	O(grado(u))	O(V)	Lista	Solo recorre los vecinos reales
Agregar arista	O(1)	O(1)	Empate	Se actualizan dos posiciones
Recorrido BFS/DFS	O(V + E)	O(V²)	Lista	La matriz revisa filas completas

4.6 Resultado Esperado

Operación	Resultado
Grados de A, B, C, D, E y F	`[2, 2, 2, 3, 2, 1]` (suma 12 = 2 × 6 aristas)
Lista de adyacencia de D	`[B, C, E]`
BFS desde A	`[A,B,C,D,E,F]`
DFS desde A	`[A,B,D,C,E,F]`
Eliminar D–E y ejecutar BFS desde A	`[A,B,C,D]`. E y F quedan sin visitar.
Componentes conexas tras eliminar D–E	2: `{A,B,C,D}` y `{E,F}`

4.7 Aplicación Práctica

Los grafos no dirigidos modelan relaciones simétricas: amistades en una red social, calles de doble sentido en un mapa o enlaces entre computadoras en una red. BFS permite encontrar la ruta con menos saltos y las componentes conexas indican qué partes de la red están aisladas.

En el proyecto del curso, un grafo no dirigido puede representar conexiones entre elementos del sistema y un BFS responde si dos de ellos están relacionados, sin recorrer todos los datos.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Los estudiantes agregan la arista en una sola lista y olvidan la otra. Hay que insistir en que en un grafo no dirigido toda arista se escribe dos veces.
- Confunden BFS con DFS o se olvidan de marcar los vértices visitados, por lo que el recorrido repite vértices o entra en un ciclo.

Recomendaciones:

- Dibujar el grafo y escribir la lista de adyacencia al lado; marcar con un color cada vértice visitado y con otro la cola o la pila.
- Mostrar en el visualizador online (visualgo.net) los recorridos BFS y DFS para que el estudiante vea el orden de visita de cada vértice.

Puntos de mayor dificultad:

- Mantener la simetría al eliminar una arista: quitarla de ambas listas.
- Entender que el orden de visita depende del orden de las listas de adyacencia, por lo que otro orden también puede ser correcto.

6. Referencias

1. GeeksforGeeks. (s. f.). Graph and its representations. https://www.geeksforgeeks.org/graph-and-its-representations/
2. Visualizador interactivo de grafos: https://visualgo.net/en/graphds
3. Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Cap. 22: Elementary Graph Algorithms.
4. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson. Cap. 9: Graph Algorithms.
5. Material de cátedra: Diapositivas EDD — Grafos no dirigidos. USAC, 2026.